

WAV2VEC2-CTC FINE-TUNING ON LIBRISPEECH WEBDATASET: STAGED OPTIMIZATION AND TRAIN-TEXT LM DECODING

Chang-Min Lee

Department of Artificial Intelligence, Korea University, Seoul 02841, South Korea
lbrightjade@korea.ac.kr

ABSTRACT

This paper presents a Wav2Vec2-CTC automatic speech recognition pipeline for the provided LibriSpeech WebDataset. The final main protocol initializes from Wav2Vec2-base, uses a newly initialized CTC classification head, and reserves one training shard for checkpoint and decoder selection. A key optimization issue is blank-dominant collapse during small-data CTC fine-tuning when the output head is randomly initialized. To address this issue, the system uses staged CTC fine-tuning: first warming up the CTC head with the Wav2Vec2 backbone frozen, and then fine-tuning the encoder while keeping the convolutional feature extractor frozen. The selected main single-model system uses beam search with a train-text trigram language model and achieves WERs of 0.246386 and 0.329289 on `test-clean` and `test-other`, respectively. An exploratory ROVER ensemble over fold-specific LM-fused hypotheses achieves the lowest observed WERs of 0.197314 and 0.271383, but it is reported separately from the main validation-selected result.

Index Terms— Automatic Speech Recognition, Wav2Vec2, CTC, Language Model Fusion, ROVER, LibriSpeech

1. INTRODUCTION

Automatic speech recognition (ASR) converts speech signals into text. This project addresses ASR on the provided LibriSpeech WebDataset by fine-tuning a Wav2Vec2-CTC model and evaluating word error rate (WER) on `test-clean` and `test-other`. The complete pipeline reads audio-transcript pairs from WebDataset tar shards, trains a CTC-based acoustic model, decodes frame-level output distributions into word sequences, and computes WER against reference transcripts. Wav2Vec2 provides the pretrained speech representation backbone [1], while connectionist temporal classification (CTC) enables transcript-level training without frame-level alignments [2]. Evaluation is reported on the provided `test-clean` and `test-other` splits, following the LibriSpeech-style ASR protocol [3].

During early development, ASR-supervised checkpoints were used only as positive-control diagnostics to verify data loading, inference, decoding, and WER computation. The final reported system instead initializes from a self-supervised Wav2Vec2 checkpoint so that the main result reflects adaptation from pretrained speech representations rather than from an already trained ASR output head.

A second design requirement is strict separation between model selection and final test evaluation. Because the provided training set is small, the project reserves one training shard for checkpoint and decoder selection, while the official test splits are used only after

the acoustic model and decoder have been fixed. This separation is important for distinguishing the main validation-selected result from exploratory extensions such as all-train model training or ROVER voting.

This setting introduces a specific CTC optimization challenge. Because the project tokenizer defines the output vocabulary, the CTC classification layer is newly initialized. CTC includes a blank symbol, and early training can assign most frames to this blank symbol, producing empty or nearly empty hypotheses. In the observed failed runs, this blank-dominant behavior led to WER close to 1.0. This indicates that the main issue was not only transcript search, but also stable random-head CTC optimization.

This paper therefore focuses on building a strict Wav2Vec2-CTC pipeline and stabilizing CTC fine-tuning from Wav2Vec2-base. The training process is examined across learning rate, freezing strategy, SpecAugment strength, and training schedule. Based on this analysis, the final acoustic recipe uses staged CTC fine-tuning with weak SpecAugment. The selected main single-model system uses beam search with a train-text trigram language model. In addition, an exploratory extension applies ROVER voting over fold-specific LM-fused hypotheses, but this extension is reported separately from the main validation-selected result [4].

2. METHOD

Figure 1 summarizes the overall system. The main reproducible path consists of staged CTC fine-tuning followed by beam decoding with a train-text trigram language model. The exploratory ensemble extension decodes multiple fold-specific acoustic models with the same LM-fused decoder and then applies ROVER voting to the resulting word hypotheses.

2.1. Data Split and Model Initialization

The main protocol uses train shards 000000–000003 for acoustic model training and reserves train shard 000004 for checkpoint selection and decoder tuning. The two official test sets, `test-clean` and `test-other`, are used only for final evaluation. The main model is initialized from Wav2Vec2-base. Since the project tokenizer defines the output vocabulary, the CTC classification layer is newly initialized before fine-tuning.

2.2. Implementation Details

Audio and transcripts are loaded directly from WebDataset tar shards. Each waveform is converted to mono when necessary and processed by the Wav2Vec2 feature extractor. Transcripts are normalized before tokenization. Training uses dynamic padding for both audio inputs and CTC labels, with padded label tokens

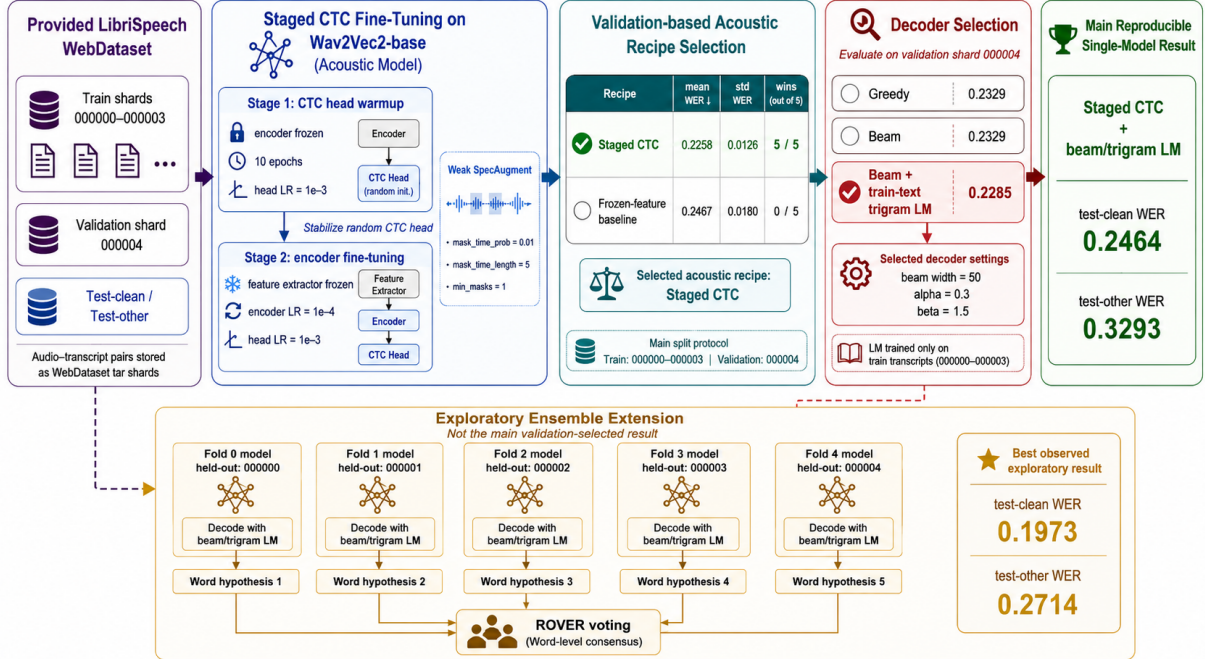


Fig. 1. Overview of the Wav2Vec2-CTC ASR pipeline. The top path shows the main validation-selected single-model protocol: staged CTC fine-tuning, acoustic recipe selection, decoder selection, and final test evaluation. The bottom path summarizes the exploratory ROVER ensemble extension, which combines fold-specific LM-fused hypotheses and is not used as the main validation-selected result.

masked as -100. The implementation uses the Hugging Face AutoModelForCTC loss [5], with the zero-infinity option enabled, gradient clipping, WER-based validation, and a finite-loss preflight check before full training. For the selected Wav2Vec2-base setting, the attention mask is not passed during the CTC loss computation, matching the implementation used for the final strict run.

2.3. Staged CTC Fine-Tuning

The selected acoustic method is staged CTC fine-tuning. This procedure separates adaptation of the newly initialized CTC classification head from broader acoustic fine-tuning. In the first stage, the Wav2Vec2 backbone is frozen and only the CTC head is trained. This head warmup stage allows the model to form an initial mapping from pretrained acoustic representations to output tokens without immediately modifying the encoder.

In the second stage, training starts from the warmed-up model. The convolutional feature extractor remains frozen, while the Transformer encoder and CTC head are fine-tuned. This design reduces the risk of early representation drift while still allowing the model to adapt to the project data. In the selected configuration, the first stage trains the CTC head for 10 epochs with learning rate 1×10^{-3} . The second stage runs for up to 40 epochs with validation-based checkpoint selection, encoder learning rate 1×10^{-4} , and head learning rate 1×10^{-3} .

2.4. Weak SpecAugment

SpecAugment is a widely used acoustic augmentation method for ASR, but the default setting is not used in the final stable

recipe [6]. In the observed small-data, random-head setting, the default SpecAugment configuration was associated with unstable training behavior, including non-finite losses or blank-dominant outputs. No-augmentation runs were therefore used as diagnostics to isolate the cause of instability. The selected weak SpecAugment configuration uses `mask_time_prob=0.01`, `mask_time_length=5`, and a minimum mask count of 1. This configuration retains modest acoustic augmentation while avoiding the instability observed with the stronger setting.

2.5. Beam Search and Train-Text LM Fusion

External language-model integration has been used to improve decoded ASR hypotheses in related end-to-end ASR settings [7]. For the main single-model system, CTC outputs are decoded using beam search with a word trigram language model trained only from transcripts in train shards 000000-000003. The train-text language model is implemented as an add- k word trigram model with $k = 0.1$ [8]. During beam decoding, the language model scores completed words using smoothed log probabilities conditioned on the previous word context. If a trigram context is unavailable, the score backs off to a shorter context. The selected decoder uses beam width 50, language-model weight $\alpha = 0.3$, and insertion bonus $\beta = 1.5$. This decoder is selected on the reserved validation shard before test evaluation.

2.6. ROVER Ensemble Extension

ROVER is used only as an exploratory ensemble extension. Each fold-specific staged CTC model is first decoded with the same beam/trigram-LM configuration. This produces one LM-fused transcript hypothesis per fold model. ROVER is then applied after

decoding by aligning the multiple word hypotheses and performing deterministic word-level voting. Thus, ROVER does not replace language-model fusion. It combines multiple LM-fused hypotheses after each model has already been decoded. Because this ensemble does not follow the same clean validation-selection protocol as the main single model, it is treated only as an exploratory extension.

3. EXPERIMENTS

3.1. Dataset and Evaluation Protocol

Table 1 summarizes the main experimental setup. The reserved validation shard is used for main checkpoint selection and decoder selection. Acoustic recipe comparison is performed separately over five train-shard folds, where each fold holds out one train shard and trains on the other four. The test sets are reserved for final WER reporting. WER is computed as

$$\text{WER} = \frac{S + D + I}{N},$$

where S , D , and I denote substitutions, deletions, and insertions, and N is the number of reference words.

Table 1. Main implementation and evaluation setup.

Item	Setting
Dataset	Provided LibriSpeech WebDataset
Main train	Shards 000000–000003
Validation	Shard 000004
Test sets	test-clean, test-other
Initialization	Wav2Vec2-base
Objective	CTC loss, zero-infinity enabled
Stage 1	CTC head warmup, backbone frozen
Stage 2	Feature extractor frozen, encoder/head tuned
SpecAugment	time prob 0.01, length 5, min 1
Decoder	Beam width 50 + train-text trigram LM
Metric	WER

3.2. Fine-Tuning Search Space

The final recipe was selected through a structured set of training and decoding diagnostics rather than a single trial. The explored factors include learning rate, freezing strategy, SpecAugment strength, training schedule, and decoding method. The acoustic comparison uses five train-shard folds, where each fold holds out one train shard and trains on the other four. It compares the selected staged CTC recipe against a frozen-feature baseline. Decoder selection is performed on the reserved validation shard by comparing greedy decoding, plain beam search, and beam search with the train-text trigram language model.

4. RESULTS AND ANALYSIS

4.1. Stability Diagnosis

Early Wav2Vec2-base CTC runs frequently produced blank-dominant hypotheses and WER close to 1.0. Table 2 summarizes the diagnostic pattern observed in these failed runs. This indicates that the main bottleneck in the observed runs was stable CTC optimization rather than decoder selection alone. In the tested settings, lowering the learning rate alone did not resolve the collapse. The stronger

SpecAugment setting was associated with additional instability, while the selected weak setting retained augmentation without reproducing the collapse.

Table 2. Observed blank-collapse diagnostic pattern in early failed runs on 56 validation utterances.

Diagnostic	Early failed runs
Validation WER	1.000
Empty-hypothesis rate	1.000 (56/56)
Blank-token rate	1.000
Nonblank-token rate	0.000

4.2. Acoustic Model Comparison

Table 3 compares the selected staged CTC recipe with a frozen-feature baseline. Staged CTC fine-tuning outperforms the baseline on all five folds and reduces mean validation WER from 0.246672 to 0.225792. This supports the two-stage design: first stabilizing the CTC head and then fine-tuning the encoder.

Table 3. Acoustic recipe comparison over five train-shard folds, each holding out one train shard.

Recipe	Mean WER	Std	Wins
Staged CTC	0.225792	0.012595	5/5
Frozen-feature	0.246672	0.017991	0/5

4.3. Decoder Selection

Table 4 shows the decoder selection result on the reserved validation shard. Plain beam search matches greedy decoding in this validation setting, while adding the train-text trigram language model reduces validation WER to 0.228487. Therefore, beam search with train-text trigram LM fusion is selected for the main single-model evaluation.

Table 4. Decoder selection on the validation shard.

Decoder	Beam	α, β	Val. WER
Greedy	–	–	0.232938
Beam	50	–	0.232938
Beam + trigram LM	50	0.3, 1.5	0.228487

4.4. Main and Exploratory Test Results

Table 5 reports the main reproducible result and exploratory extensions. The main single model denotes the validation-selected staged CTC system with beam/trigram-LM decoding. The all-train model is an exploratory model trained on all five train shards. The ROVER ensemble is the best observed exploratory extension and combines fold-specific LM-fused hypotheses.

Table 5. Test WER results. The main result is the validation-selected single model; the other systems are exploratory.

System	Role	clean	other
Main single model	Main	0.246386	0.329289
All-train model	Expl.	0.216867	0.303307
ROVER ensemble	Expl.	0.197314	0.271383

The main single-model pipeline achieves WERs of 0.246386 and 0.329289 on `test-clean` and `test-other`, respectively. The all-train model obtains lower WERs when trained on all five train shards, but it lacks an untouched in-domain validation shard. The fold ensemble with ROVER voting gives the lowest observed WERs, 0.197314 and 0.271383, but it is reported separately as an exploratory ensemble extension because folds 0–3 include shard 000004 among their training shards, while shard 000004 is the reserved validation shard for the main single-model protocol.

4.5. Discussion

The results suggest that stable random-head CTC optimization was the main bottleneck in the observed experiments. Decoder tuning alone did not resolve blank-dominant collapse in the early failed runs. After staged CTC produced usable hypotheses, train-text LM fusion provided a modest validation improvement. ROVER achieved lower observed WERs in the exploratory setting by aggregating multiple fold-specific LM-fused hypotheses. However, this result should not be interpreted as the clean main result because the ensemble does not follow the same validation-selection protocol as the main single model.

5. CONCLUSION

This paper presents a Wav2Vec2-CTC ASR pipeline for the provided LibriSpeech WebDataset. The final main system initializes from Wav2Vec2-base with a newly initialized CTC head and uses a validation-selected staged fine-tuning protocol. The main observed challenge is blank-dominant collapse in small-data random-head CTC fine-tuning, which is mitigated through CTC-head warmup, encoder fine-tuning with a frozen feature extractor, and weak SpecAugment. The main reproducible single-model pipeline uses beam/trigram-LM decoding and achieves WERs of 0.246386 and 0.329289 on `test-clean` and `test-other`. An exploratory ROVER ensemble achieves the lowest observed WERs of 0.197314 and 0.271383, but it is reported separately from the main validation-selected result. Future work includes stronger language models, cleaner ensemble validation, larger ASR backbones, and systematic ensemble-diversity analysis.

6. REFERENCES

- [1] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in *Advances in Neural Information Processing Systems*, 2020, vol. 33, pp. 12449–12460.
- [2] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the International Conference on Machine Learning*, 2006, pp. 369–376.
- [3] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing*, 2015, pp. 5206–5210.
- [4] Jonathan G. Fiscus, “A post-processing system to yield reduced word error rates: Recognizer output voting error reduction,” in *Proceedings of the IEEE Workshop on Automatic Speech Recognition and Understanding*, 1997, pp. 347–354.
- [5] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [6] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Proceedings of Interspeech*, 2019, pp. 2613–2617.
- [7] Jan Chorowski and Navdeep Jaitly, “Towards better decoding and language model integration in sequence to sequence models,” *arXiv preprint arXiv:1612.02695*, 2016.
- [8] Stanley F. Chen and Joshua Goodman, “An empirical study of smoothing techniques for language modeling,” in *Proceedings of the 34th Annual Meeting of the Association for Computational Linguistics*, 1996, pp. 310–318.